

Cryptography: an introduction

Cataldo Basile

< cataldo.basile @ polito.it >

Politecnico di Torino

Dip. Automatica e Informatica

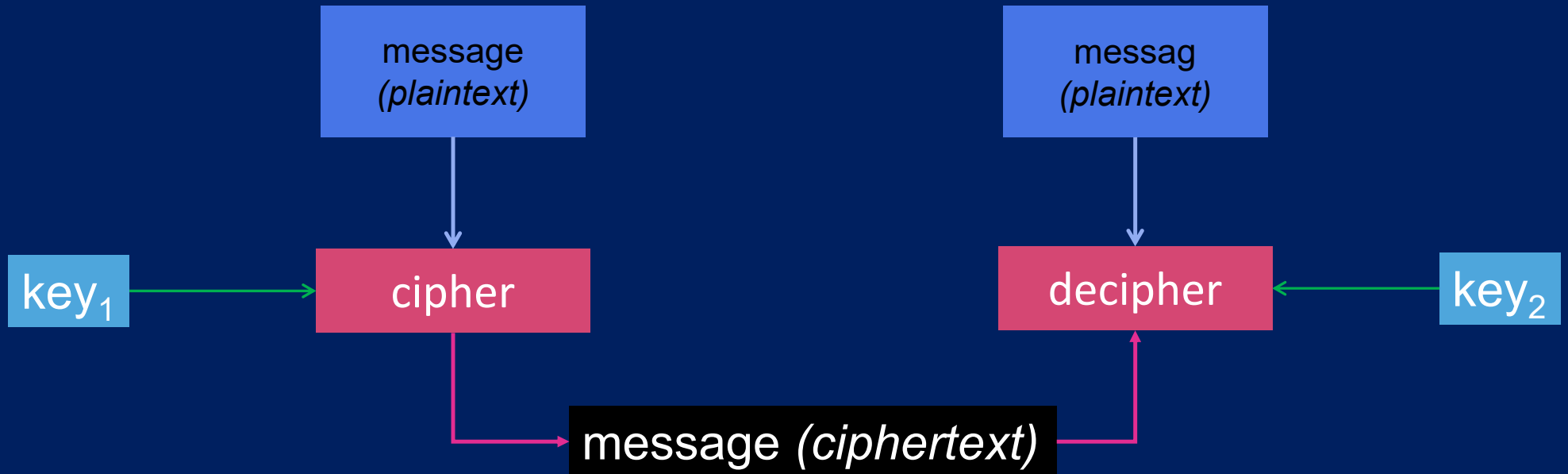
Agenda

- introduction to symmetric crypto
 - definitions
 - algorithms and keys
 - stream algorithms
 - block algorithms
- hash functions and digests
 - collisions
 - data authentication
 - keyed digest
 - authenticated encryption

Randomness and pseudo randomness

- true random numbers and sequences
 - generated with random processes
 - physical processes: white noise, Brownian processes, ...
- pseudo-random numbers and pseudo-random sequences
 - generated with a deterministic process
 - ...statistically indistinguishable from true random numbers/sequences
 - require sources of entropy to start
 - if the processes are weak, sequences are predictable
 - pseudo-random generators can only generate a finite number of sequences
 - imagine them numbered → associated them with an integer number
 - this integer is named *seed*

Cryptography as a transformation



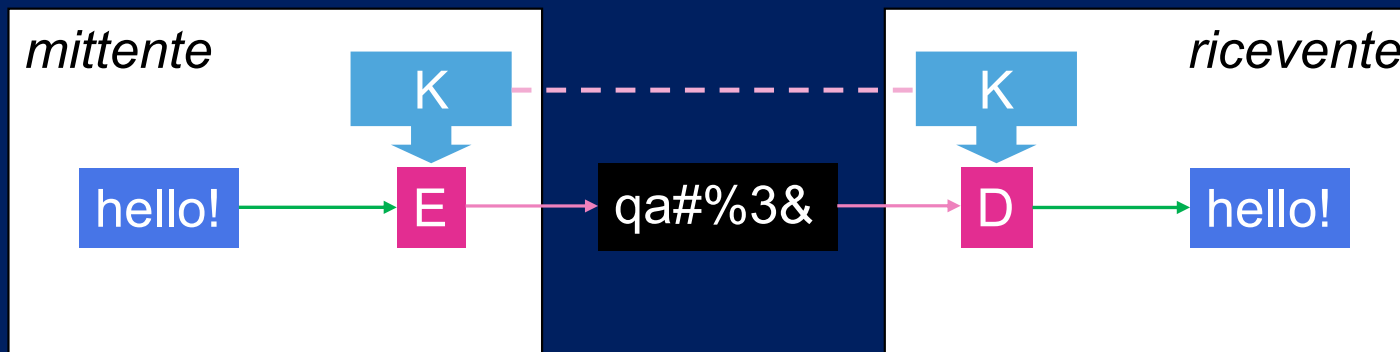
Why do we need randomness?

- processes: select a random number and keep it secret
 - e.g. a cryptographic key
 - if an attacker can predict the key or part of it, it will not hard to guess the key I used
- effects: the result of a cryptographic operation must “look random”
 - so attackers cannot gain any knowledge from it

Symmetric cryptography

[Secret key cryptography]

- the **same key** is shared between sender and receiver
- (quite) low computational effort → best for encrypting large quantities of data
 - $C = \text{enc}(K, P)$ or $C = \{P\}_K$
 - $P = \text{dec}(K, C) = \text{enc}^{-1}(K, C)$
- problem: how to share the key securely?



Caesar cipher

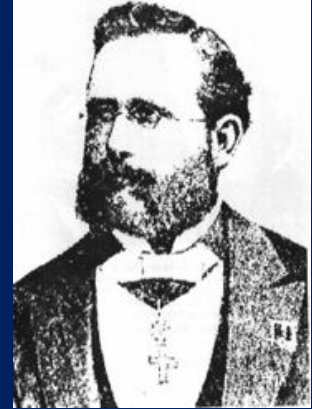
- the first known encryption algorithm
 - associates every letter with another selected next in the alphabetical order
- weak and obsolete!
 - just interesting historically...

Security through obscurity (STO)

- principle that states that something is secure just because it is kept secret (algorithms, methods, ...)
 - clearly just an illusion...
 - Hobbs defined this principle as useless in 1851
- **still widespread**
 - ...and the reason for several attacks
 - Shannon: “The enemy knows the system.”



Cryptography strenght (*Kerckhoffs principle*)



- if the keys:
 - are kept secret
 - are managed only by trusted systems
 - are of adequate length
- then ...
- ... it has no importance that the encryption and decryption algorithms are kept secret
- ... on the contrary, it is better to make the algorithms public so that they can be widely analysed and their possible weaknesses identified

Auguste Kerckhoffs, "La cryptographie militaire", Journal des sciences militaires, vol. IX, pp. 5–38, Janvier 1883, pp. 161–191, Février 1883.

Lengths of the secret keys

- if:
 - the encryption algorithm is well designed
 - the keys, n bits long, are kept secret
- ... then the only possible attack is the brute force (exhaustive) attack
 - which requires a number of trials equal to

$$T = 2^n$$

Encryption algorithms

- AES
 - AES 128, AES 192, AES 256
- DES
 - 3DES
- RC2, RC4
- CHACHA20 (SALSA20)
- ARIA, SEED, SM4, CAMELLIA, CAST

Forza degli algoritmi simmetrici

- depends on the time needed to perform all the trials (brute force)
- ...if the algorithms are well designed
 - compared to the time available to the attackers
 - context dependent (e.g., hours, days, years)
- the longer the stronger (using the same algorithm)
 - $2^{40}=1099511627776$
 - $2^{256}=115792089237316195423570985008687907853269984665640564039457584007913129639936$
- account for each tried key
 - time to actually execute the decryption +
 - time to evaluate if you guessed the key

Algorithms become obsolete

- time to guess keys reduces by evolution of the technology
 - algorithms will sooner or later become weak
 - computers can perform many more trials in the same amount of time
 - brute force can be easily parallelized
 - e.g., GPUs with hundred cores
 - DES used a 56-bit key and is definitely obsolete
- cryptanalysis
 - theoretical and statistical techniques that, based on observations, reduce the search space
 - DES: 56 $\rightarrow$ $\sim$ 40bit

Keys expire and need to be refreshed

- even if I choose them of the proper lenght
 - lenght $\geq$ 128 bit
- time reasons
 - an attacker had enough time to test a considerable number of keys
- use reasons
 - the more the date encrypted the more the info cryptanalysts obtained
- when you renew them, choose random ones
 - no need to brute force them if I can guess them or predict several bits out of them

Stream vs. block ciphers

Booleana EX-OR (XOR)

- ideal “confusion” operator
- if the input is random (probability 0 : 1 = 50 : 50%) then also the output will be equally random
- primitive operation available on all CPU
- truth table

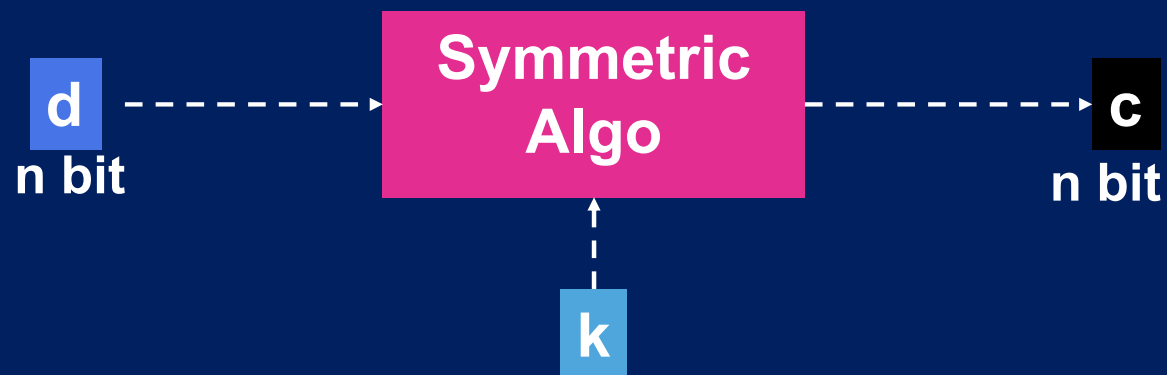
$\oplus$	0	1
0	0	1
1	1	0

Algoritmi simmetrici di tipo stream

- operate on a stream of data without requiring the division in blocks, typically on one bit or one byte at a time
- ideal algorithm
 - one-time pad (requires a key which is as long as the message to protect!)
- real algorithms
 - pseudo-random generators where the key is used as a seed
 - every time I use the same seed, the algorithm generates the same sequence
 - bitwise XOR between the plaintext and the sequence
 - (old) RC4, SEAL
 - (modern ma not state of the art) Salsa20
 - (state of the art) ChaCha20

Algoritmi a blocchi

- transform one block of data at a time
 - fixed size
 - 64 bit = obsolete algorithms
 - 128 bit = recent ones
- the key determines the actual transformation
 - used to decide what and how to transform plaintext into ciphertext

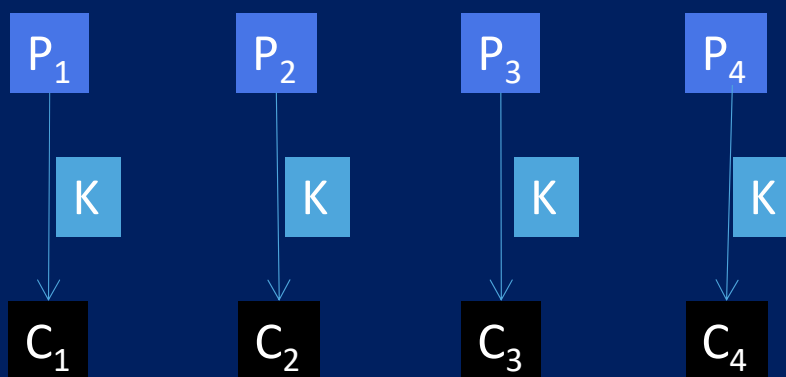


Applicazione degli algoritmi a blocchi

- algorithms only indicate how to manipulate a single block
- the **modes of operation** allow
 - ciphering **more data** than a single block
 - ECB (Electronic Code Book)
 - CBC (Cipher Block Chaining)
 - ciphering **less data** than a block
 - padding, cipher stealing
 - simulate stream algorithms using block ciphers
 - CFB (Cipher FeedBack), OFB (Output FeedBack)
 - cipher data and number blocks
 - CTR (Counter mode)

ECB (Electronic Code Book)

- formula for the i-th block:
 $C_i = \text{enc} (K, P_i)$
- NOT to be used on long messages because
 - swapping of two blocks of ciphertext goes undetected
 - identical blocks generate identical ciphertexts hence it is vulnerable to *known-plaintext* attacks



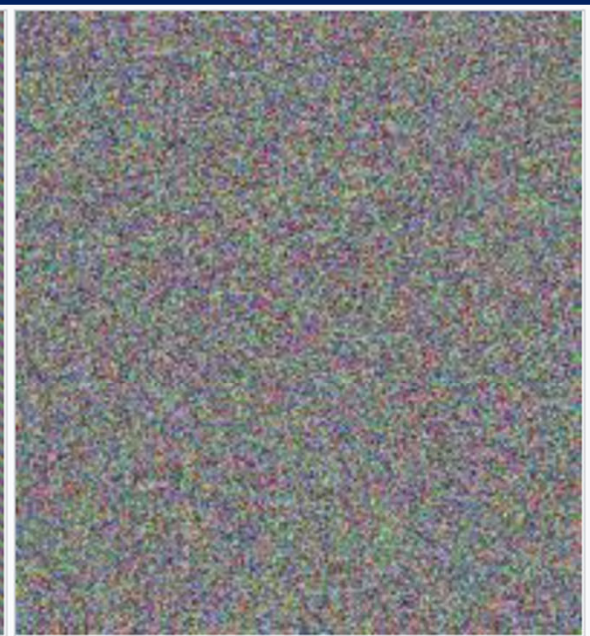
ECB: insecurity



Original image



Encrypted using ECB mode

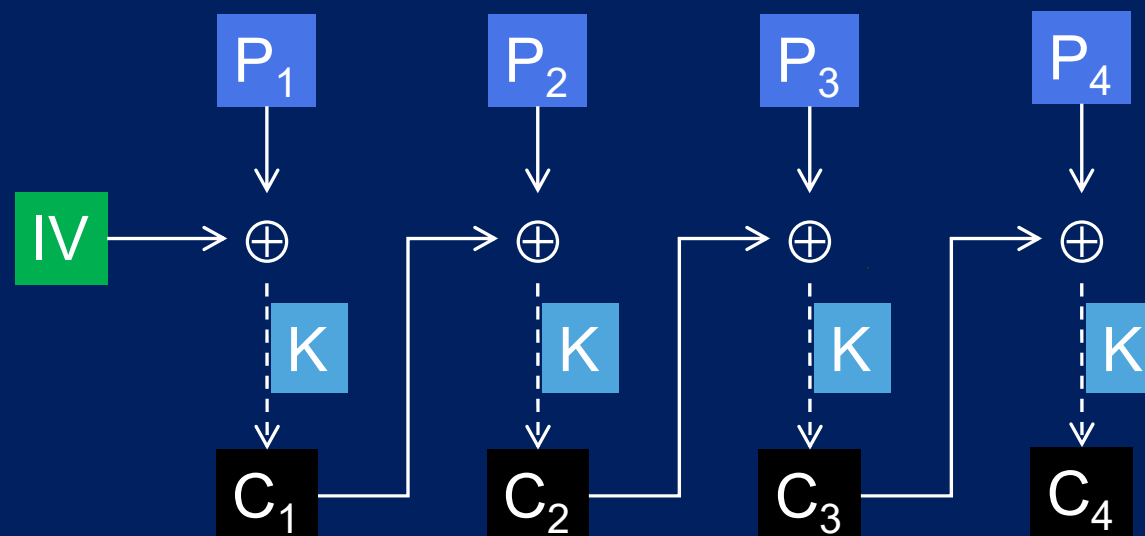


Modes other than ECB result in pseudo-randomness

https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation

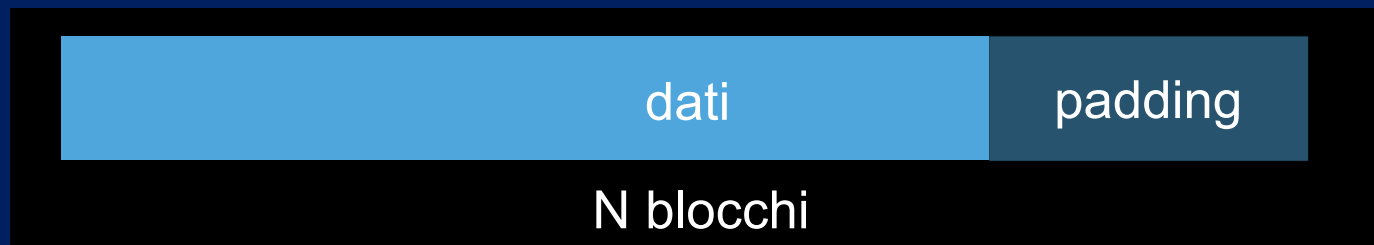
CBC (Cipher Block Chaining)

- formula for the i-th block:
 $C_i = \text{enc}(K, P_i \oplus C_{i-1})$
- requires $C_0 = \text{IV}$ (Initialization Vector)



Paddin`g

- size of algorithm's block B
- size of data to process D (not a multiple of B)
- add bits until a multiple of B is reached



- **problems:**
 - transmit/store more data (B) than needed (D)
 - value of padding bits?



Integrity...

Integrity check: high-level approach

- the sender uses a function to generate evidence of how data are made when he/she was sure they were correct
 - e.g. before sending both, data and evidence
- the recipient uses the same function on the data in his possession
 - e.g. the just received ones
- compare
 - if the function returns the same value (sender-side and recipient-side)
 - received data are correct
 - otherwise, they don't satisfy the integrity principle
- how to perform the comparison?
 - using secure algorithms and sharing the evidence in secure ways!

Message digest and hash functions

- the message digest is a **fixed-length** “summary” of the message to be protected (of any length)
- it must be
 - fast to compute
 - impossible or very difficult to invert
 - difficult to create “collisions”
- digest often used to avoid performing operations on the whole message, especially when the message is very large
 - e.g. because public-key cryptography is very slow
- digest can be calculated in many ways, but usually via a **(cryptographic) hash function**

Lunghezza del digest

- if the algorithm is well-designed
 - then all digests are pseudo-random
- if I change one bit of the input message
 - on average, 50% of the bits in the digest change
- it is always possible to generate messages with the same digest
 - If the algorithm generates an n -bit digest, then:
 - to find a message that has the same digest as a fixed message, 2^n attempts are needed
 - to find two messages with the same digest, $2^{(n/2)}$ attempts are needed
- digests with many bits are required to make collisions statistically negligible
 - said to have n bits of security if on average 2^n attempts are needed to carry out the easiest attack
- a hash function is considered valid if it has a security strength of at least 80 bits

Cryptography hash algorithms

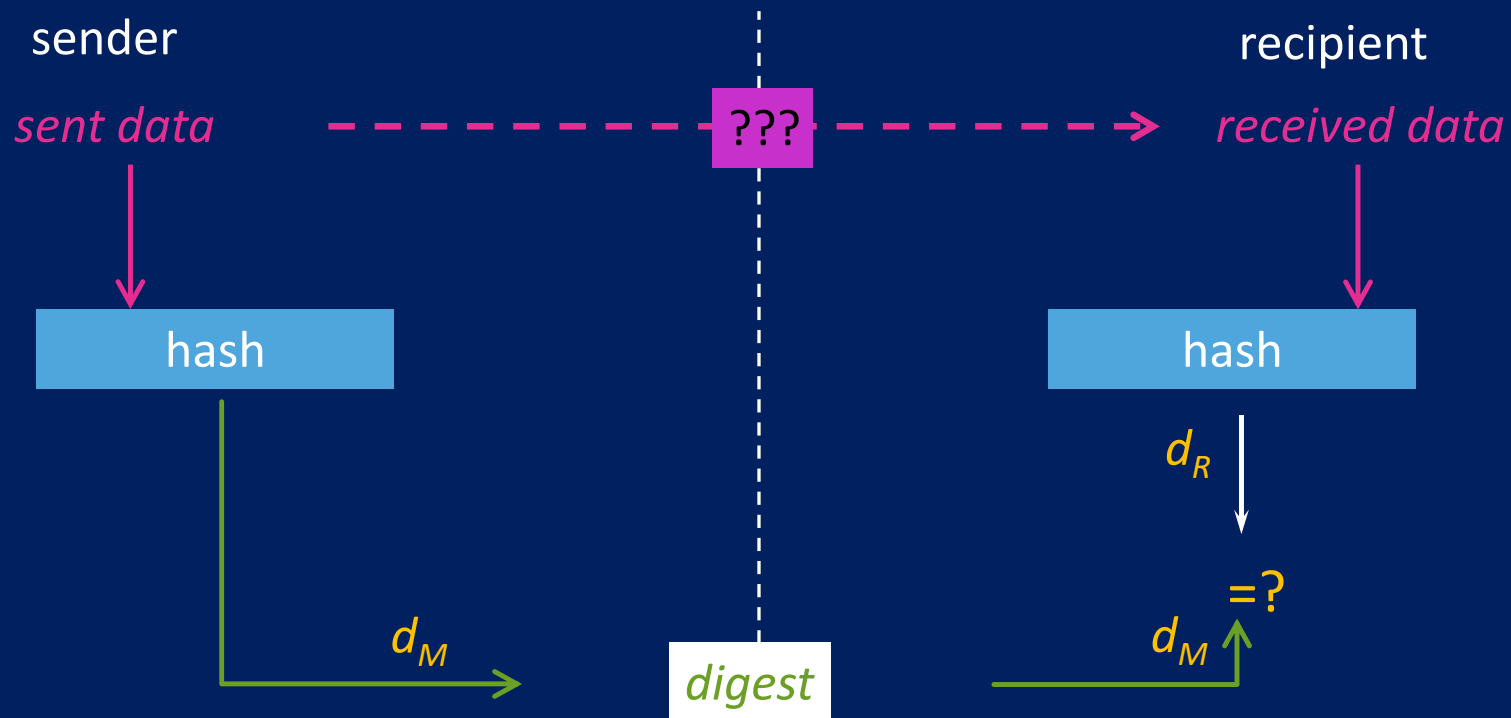
<i>name</i>	<i>block</i>	<i>digest</i>	<i>defined in</i>	<i>note</i>
MD2	8 bit	128 bit	RFC-1319	obsolete
MD4	512 bit	128 bit	RFC-1320	obsolete
MD5	512 bit	128 bit	RFC-1321	obsolete
RIPEMD-160	512 bit	160 bit	ISO/IEC 10118-3	good?
SHA-1	512 bit	160 bit	FIPS 180-1	maybe
SHA-224	512 bit	224 bit	FIPS 180-2 FIPS 180-3	good
SHA-256	512 bit	256 bit		
SHA-384	512 bit	384 bit		
SHA-512	512 bit	512 bit		
SHA-2				
SHA-3	1152-576	224-512	FIPS 202 FIPS 180-4	excellent

SHA-2 e SHA-3

- quick remediation when a serious attack was discovered against SHA-1
- define the SHA-2 family (based on the same math as SHA-1):
 - $\text{SHA-2} = \{\text{SHA-224}, \text{SHA-256}, \text{SHA-384}, \text{SHA-512}\}$
 - SHA-256 larger block = 32 bit
 - SHA-512 larger block = 64 bit
 - SHA-224/-384 = troncamento di SHA-256/-512
- **SHA-3 selected through a competition**
 - completely new scheme
 - winner **Keccak** (“catch-ack”)
 - the best one available nowadays

Ma il digest serve a qualcosa?

- not against man-in-the-middle attackers



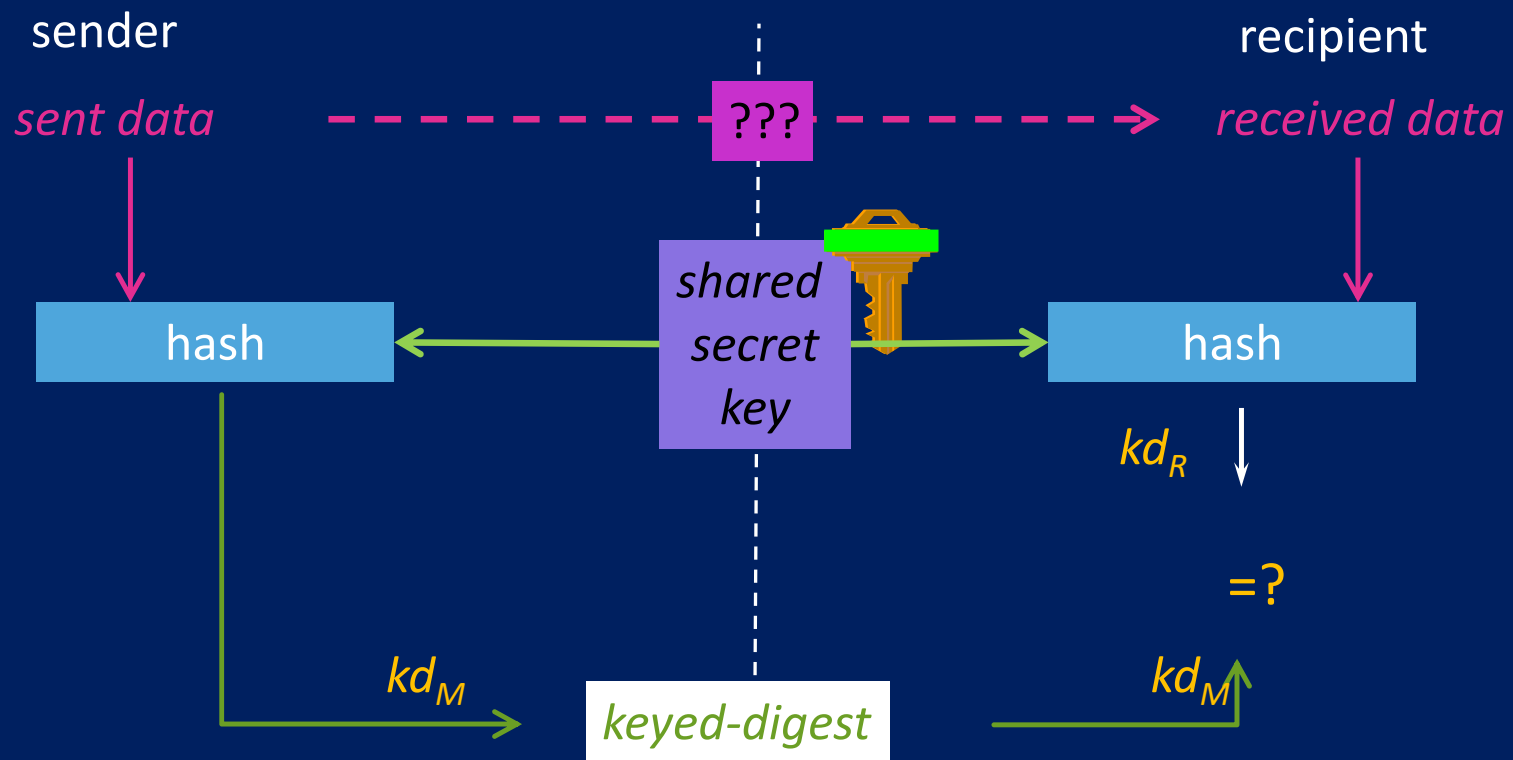
Authentication based on secrets knowledge

- dimostro di essere un determinato utente
 - dimostrando di conoscere un segreto che
 - solo io posso conoscere, oppure
 - condivido con altre persone/utenti
- in crittografia le chiavi sono tenute segrete
 - posso usarle per autenticazione dei dati
- si invia anche un digest calcolato non solo sui dati ma anche su un segreto (chiave)
 - keyed digest o Message Authentication Code (MAC)

Authentication based on knowledge

- prove that you are a specific user
 - by demonstrating that you know a secret that only you can know
 - or you share with other people/users involved in the authentication
- in cryptography, keys are kept secret
 - so can be used for data authentication
- a digest can also work
 - calculated not only on the data but also on a secret (key)
 - called a keyed digest or Message Authentication Code (MAC)

Keyed-digest



Keyed-digest: HMAC

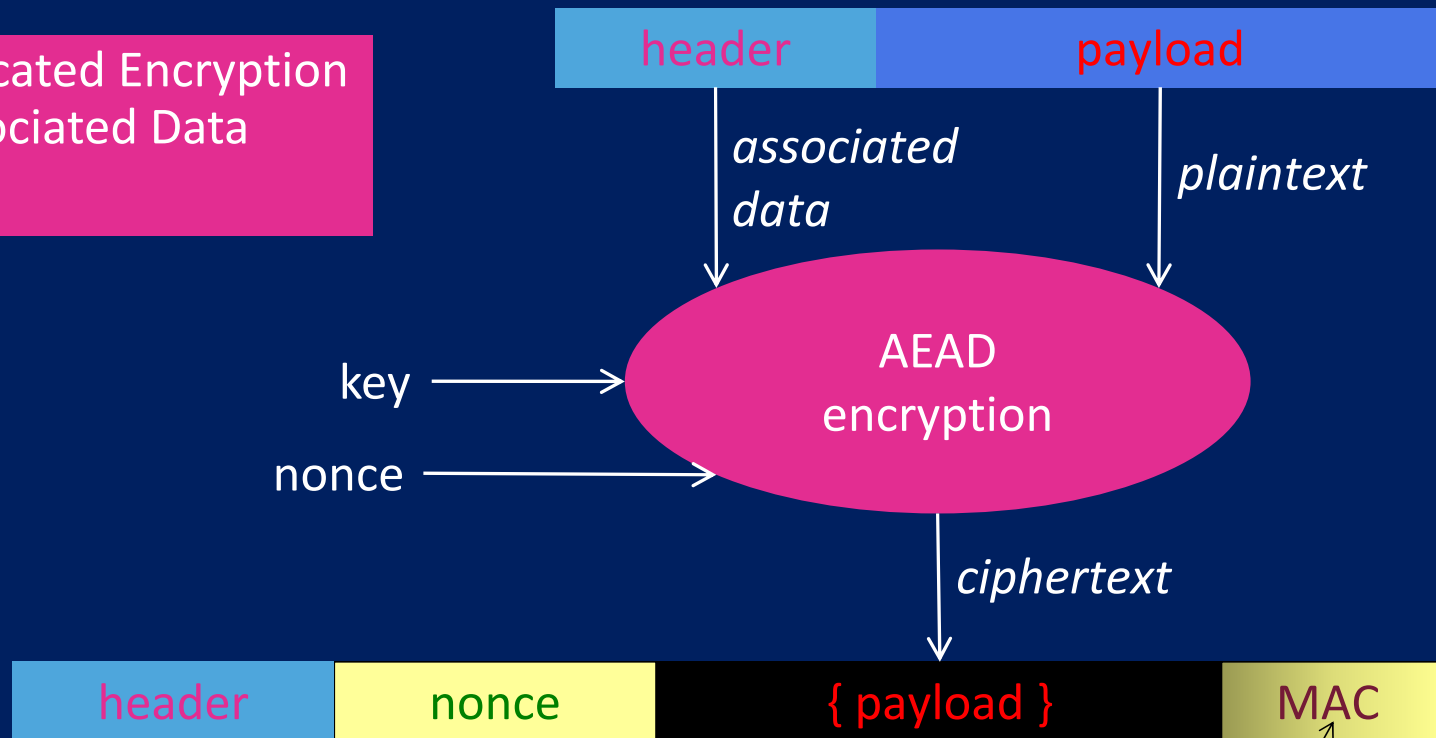
- RFC-2104 (also FIPS-198)
- base hash function H :
 - B byte block, L byte output, with $B > L$
- definitions:
 - $\text{ipad} = 0x36$ repeated B times
 - $\text{opad} = 0x5C$ repeated B times
 - deprecated keys s.t. $|K| < L$
 - if $|K| > B$ then $K' = H(K)$ else $K' = K$
 - if $|K'| < B$ then K' is 0-padded up to B bytes
- $\text{hmac} = H(K' \oplus \text{opad} || H(K' \oplus \text{ipad} || \text{data}))$

Authenticated Encryption (AE)

- one single operation for privacy and authentication (and integrity):
 - just one key and one algorithm
 - better speed
 - less error likelihood in combining the two functions
- needed by applications (e.g. e-mail messages, network packets)
- GCM the most used one
 - used by TLS
 - during the encryption
 - generates ciphertext + authentication tag
 - fast on present CPUs

Authenticated Encryption

Authenticated Encryption
with Associated Data
(AEAD)



name TAG
may be hidden in the encrypted payload